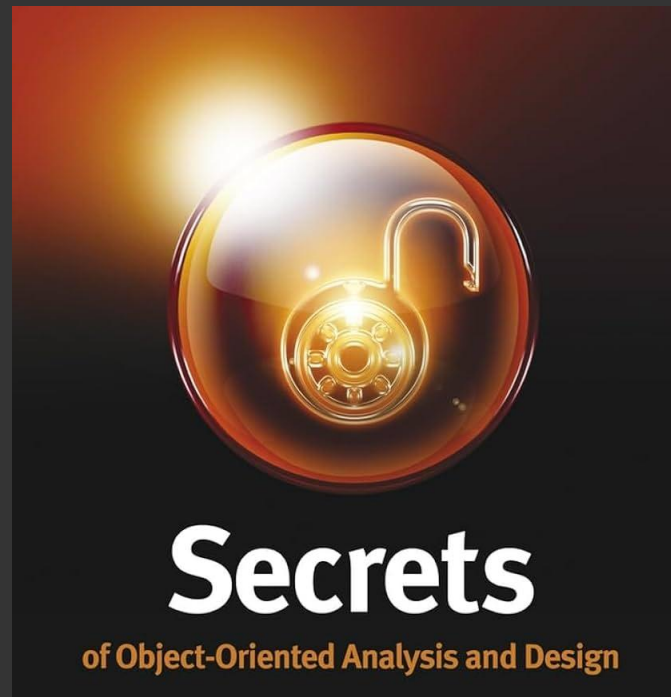


Object Oriented Analysis and Design

Vladimir (Vova) Soroka
CTO, Sandbox.co, theMind
VladimirSoroka@shenkar.ac.il

With gratitude to Yehonatan Alfasi,
Founder & CEO at Ti-Space Career
Accelerator



Vladimir (Vova) Soroka

01

Entrepreneur

CTO of Sanbdx,
theMind and Stardst,
Investor in Apolo

02

Researcher

15 years in IBM Research
B.Sc. CS Technion
MA, Communication, University of
Haifa

03

Developer

Internet Pioneer,
software developer and
manager in multiple
companies

04

Family Man

4 kids, 3 cats and 2
dogs, 1 wife



Our goals

- Deeper understanding of OOP core principles
- Deal with complicated system development
- Write software that is flexible and robust
- Learn New development methodologies
- Implement various design patterns

Why are we here?

- Upgrade our coding skills
- Design our system with future compatibility
- Learn new development techniques
- Feel confidence when coding
- Get ready for interviews

Expectations

- Attendance - For you!
- Be on time - For you!
- Ask Questions
- Help each other

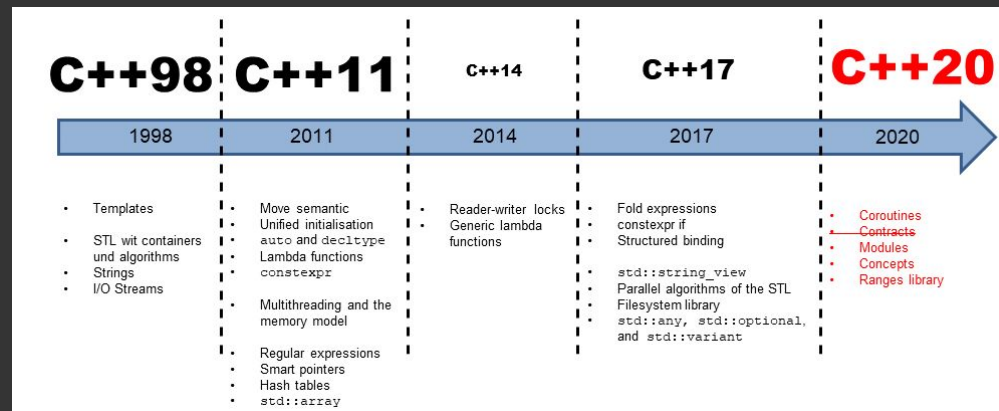


How we are going to pass this course?

Course Requirements and Grading			
Course Requirements	At least 60 in both the project and the final exam		
Grading	Project	40%	same grading weight for each step
	Assignments	10%	same grading weight for each exercise
	Quizzes	10%	same grading weight for each quiz there is no second chance in quizzes
	Exam	40%	

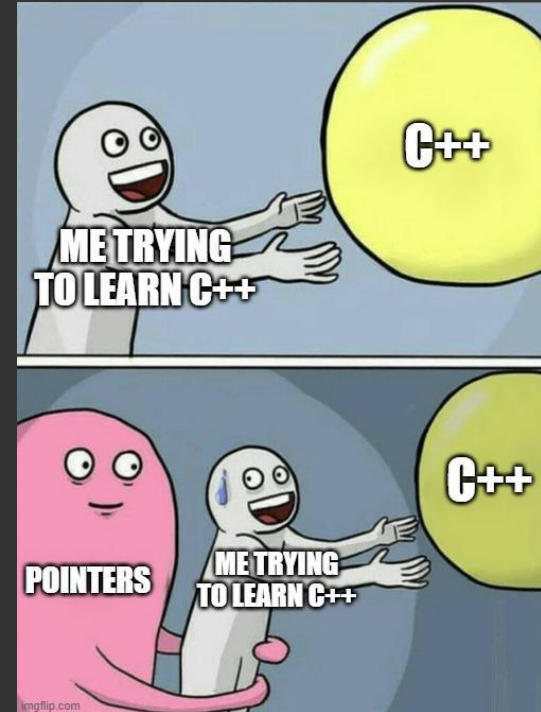
Quick follow up - OOP3

- Classes & Objects
- Constructors
- Destructors
- Class Members
- Class Methods
- Instantiation



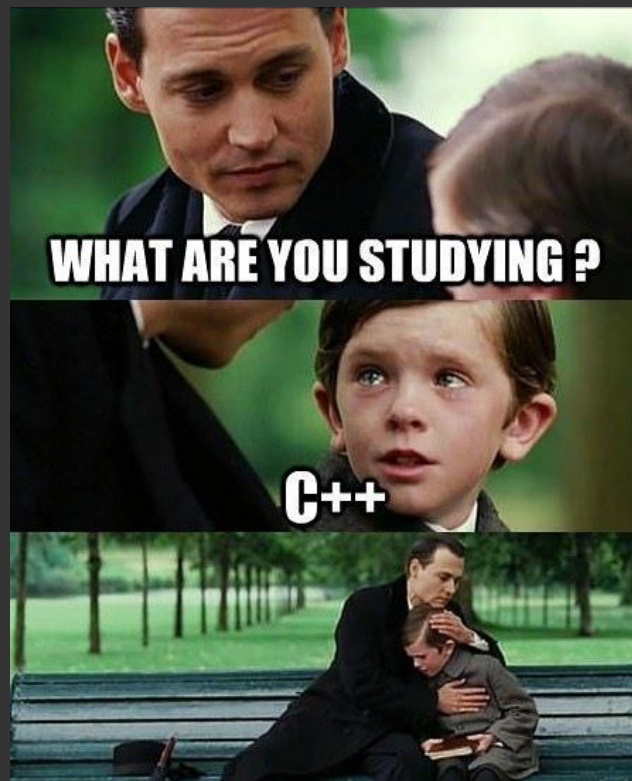
Quick follow up - OOP3

- Template functions & classes
- Files - Textual and Binary
- Polymorphism
- Inheritance
- Operators
- Exceptions
- Pointers manipulation
- Basic STL's



More STL

- Stack
- Vector
- Hash map
- Use of iterators



Auto keyword

- Auto keyword can be used to determine the variable type without explicitly write it.
- Most in use with **iterators** and long writing data types.
- `std::map<int, string>::iterator it = m.begin()`
- `auto it = m.begin()`

Lambda Functions

What are Lambda Functions?

- A **lambda function** is an **anonymous function** (a function without a name) that can be defined **inline**.
- Introduced in **C++11** to simplify function definitions and improve code readability.
- Useful for **short, simple functions**, especially in **STL algorithms**.



Lambda Functions

Lambda Function Syntax:

```
[capture] (parameters) -> return_type  
{ function_body };
```

Capture: Specifies which variables from the surrounding scope should be available inside the lambda.

Parameters: The arguments that the lambda takes (like a normal function).

Return Type (optional): If omitted, it's inferred automatically.

Function Body: The actual implementation of the lambda.



Lambda Functions – Basic Example

```
#include <iostream>

int main() {
    auto sayHello = []() { std::cout << "Hello, Lambda!" << std::endl; };
    sayHello(); // Call the lambda
    return 0;
}
```

Lambda Functions —

With parameters and return type

```
#include <iostream>

int main() {
    auto add = [](int a, int b) -> int { return a + b; };
    std::cout << "Sum: " << add(3, 4) << std::endl;
    return 0;
}
```

Lambda Functions – Capturing Variables

```
#include <iostream>

int main() {
    int x = 10;
    auto multiply = [x](int y) { return x * y; };
    std::cout << "Result: " << multiply(5) << std::endl;
    return 0;
}
```

Lambda Functions – STL Usage

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> nums = {1, 2, 3, 4, 5};
    std::for_each(nums.begin(), nums.end(), [](int n) {
std::cout << n * n << " "; });
    return 0;
}
```

Let's See Some Code

Stack, Vectors, Maps and Lambda
Functions (and some more things)